

Profiling CUDA Kernels with NVIDIA Nsight

A practical workflow for Nsight Systems and Nsight Compute

NVIDIA's profiling toolkit splits into two complementary tools. Using the right one for the question you're asking saves a lot of wasted profiling runs.

Tool	Scope	Use it to answer
Nsight Systems <code>nsys</code>	Whole-application timeline: CPU, GPU, memory copies, kernel launches, API calls, streams.	"Where does my program spend time? Are kernels overlapping with copies? Is the GPU idle waiting on the CPU?"
Nsight Compute <code>ncu</code>	Single-kernel deep dive: instruction mix, memory throughput, occupancy, warp stalls, roofline.	"Why is <i>this specific kernel</i> slow? Is it memory- or compute-bound? Are my warps stalling?"

The standard workflow is top-down: run `nsys` first to find which kernel dominates wall-clock time, then run `ncu` on just that kernel to understand why. Profiling everything with `ncu` up front is slow and buries the signal.

1. Setup

Both tools ship with the CUDA Toolkit. Verify they're on your path:

```
nsys --version
ncu --version
```

Compile with line information so the profiler can map results back to your source. Keep your optimization flags; `-lineinfo` does not disable optimization the way `-G` (full debug) does.

```
# -lineinfo: source correlation, safe for profiling optimized builds
nvcc -lineinfo -Xcompiler -fPIC -shared \
    -o geneig_warp_red_kernel.so geneig_warp_red_kernel.cu
```

Avoid `-G` for performance profiling. The full-debug flag disables compiler optimizations and changes register allocation, so the numbers you measure won't reflect your release build. Use `-lineinfo` instead.

2. Nsight Systems — find the bottleneck

Capture a timeline

```
# Profile a Python driver that launches your kernels via ctypes/CuPy
nsys profile -o my_report --stats=true python pipeline.py
```

This produces `my_report.nsys-rep` and prints a summary table to the terminal. The `--stats=true` flag gives you a quick per-kernel and per-API breakdown without opening the GUI.

Useful flags

Flag	Effect
<code>-t cuda, nvtx, osrt</code>	Trace CUDA, NVTX ranges, and OS runtime calls. Add <code>cublas / cudnn</code> if used.
<code>--stats=true</code>	Print summary tables to stdout after the run.
<code>-o name</code>	Output report file name.
<code>--force-overwrite=true</code>	Overwrite an existing report.
<code>-c cudaProfilerApi</code>	Only capture between <code>cudaProfilerStart/Stop</code> calls — skips warmup.

Reading the result

Open the report in the GUI (`nsys-ui my_report.nsys-rep`) and look for:

- **GPU gaps** — stretches where the GPU row is empty mean it's idle, usually waiting on the CPU or a blocking sync. Your `sync_device()` calls show up here.
- **Memcpy vs. compute overlap** — Host-to-Device copies running on a separate row from kernels means good overlap; serialized bars mean you're copy-bound.
- **Launch frequency** — many tiny kernel bars back-to-back suggests launch overhead dominates; consider fusing or batching.

Annotate with NVTX. Wrapping phases of your pipeline in named ranges makes the timeline readable. In Python with CuPy: `cupy.cuda.nvtx.RangePush("make_small_blocks") ... RangePop()`. The labels appear as a dedicated row in the timeline.

3. Nsight Compute — understand a single kernel

Profile one kernel by name

Once `nsys` tells you which kernel dominates, target it directly. Profiling every kernel launch is slow, so filter by name and limit launch count:

```
# -k matches kernel name (regex); -c limits how many launches to capture
ncu -k two_level_warp_reduction -c 3 \
    -o kernel_report \
    python pipeline.py
```

Templated kernels: your `two_level_warp_reduction<N_EIG>` instantiations appear as separate mangled names. A regex like `-k "two_level_warp_reduction"` matches all of them; use `--launch-skip` to jump past warmup launches and profile a steady-state one.

Target a specific metric set

```
# Full detail (slow — replays the kernel many times)
ncu --set full -k two_level_warp_reduction -c 1 -o full_report python pipeline.py

# Just the speed-of-light + memory summary (fast)
ncu --section SpeedOfLight --section MemoryWorkloadAnalysis \
    -k two_level_warp_reduction -c 1 python pipeline.py
```

Key sections to read

Section	What it tells you
GPU Speed of Light	Top-level: % of peak compute vs. % of peak memory bandwidth. The higher one is your bound.
Memory Workload Analysis	Throughput per memory level, and whether global loads are coalesced.
Occupancy	Achieved vs. theoretical occupancy, and the limiter (registers, shared memory, or block size).
Scheduler / Warp State	Why warps stall — memory dependency, barrier (<code>__syncthreads</code>), or execution dependency.
Source Counters	With <code>-lineinfo</code> , maps stalls and instruction counts to individual source lines.

4. Interpreting the numbers

Memory-bound vs. compute-bound

The Speed of Light section is the first thing to read. If memory throughput is near peak and compute is low, you're **memory-bound** — optimize access patterns, coalescing, and data reuse before touching arithmetic. If compute is near peak and memory is low, you're **compute-bound** — reduce instruction count or improve ILP. A reduction kernel like a warp-reduction outer product is typically memory-bound on the input read.

Occupancy isn't the goal

Higher occupancy is not automatically faster. A register-heavy kernel (like a templated kernel that holds an `acc[N_SYM]` array in registers) trades occupancy for fewer memory round-trips, and that's often a win. Use occupancy to explain a latency-bound symptom, not as a target to maximize blindly.

Common warp-reduction findings

- **Uncoalesced loads** on the diffuse/noise arrays show up as low global-load efficiency — check your indexing stride against thread index.
- **Barrier stalls** at `__syncthreads()` between the per-warp and cross-warp reduction phases are expected; large values suggest imbalanced work per warp.

- **Shared memory bank conflicts** in the `sh[i * num_warps + warp]` write pattern — Nsight Compute reports conflicts per request directly.

5. Quick command reference

Goal	Command
Whole-app timeline + stats	<code>nsys profile --stats=true -o rep python pipeline.py</code>
Open timeline GUI	<code>nsys-ui rep.nsys-rep</code>
Profile one kernel, fast	<code>ncu --set basic -k NAME -c 1 python pipeline.py</code>
Profile one kernel, full	<code>ncu --set full -k NAME -c 1 -o rep python pipeline.py</code>
Open kernel report GUI	<code>ncu-ui rep.ncu-rep</code>
Skip warmup launches	<code>ncu --launch-skip 5 -c 1 ...</code>

Remote / cluster note: on an HPC node you'll typically run `nsys / ncu` headless over SSH to produce the `.nsys-rep / .ncu-rep` file, then copy it back and open it in the local GUI. `ncu` needs elevated GPU performance-counter permissions; if you get a `ERR_NVGPUCTRPERM` error, the cluster admin must enable counter access (or run via a profiling-enabled allocation).